

Experiment # 07

Infix, Prefix and Postfix

Objectives:

The objective of this lab is to understand the conversions between infix to prefix and postfix

- Infix to prefix
- Infix to postfix
- Prefix to postfix and vice versa

Software Tools:

- Microsoft Visual Studio

Theory:

The way to write arithmetic expression is known as a notation. An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression. These notations are –

- Infix Notation
- Prefix (Polish) Notation
- Postfix (Reverse-Polish) Notation

These notations are named as how they use operator in expression. We shall learn the same here in this chapter.

Infix Notation

We write expression in infix notation, e.g. $a - b + c$, where operators are used in-between operands. It is easy for us humans to read, write, and speak in infix notation but the same does not go well with computing devices. An algorithm to process infix notation could be difficult and costly in terms of time and space consumption.

Prefix Notation

In this notation, operator is prefixed to operands, i.e. operator is written ahead of operands. For example, $+ab$. This is equivalent to its infix notation $a + b$. Prefix notation is also known as Polish

Notation.

Postfix Notation

This notation style is known as Reversed Polish Notation. In this notation style, the operator is postfix to the operands i.e., the operator is written after the operands. For example, $ab+$. This is equivalent to its infix notation $a + b$.

The following table briefly tries to show the difference in all three notations –

Sr.No.	Infix Notation	Prefix Notation	Postfix Notation
1	$a + b$	$+ a b$	$a b +$
2	$(a + b) * c$	$* + a b c$	$a b + c *$
3	$a * (b + c)$	$* a + b c$	$a b c + *$
4	$a / b + c / d$	$+ / a b / c d$	$a b / c d / +$
5	$(a + b) * (c + d)$	$* + a b + c d$	$a b + c d + *$
6	$((a + b) * c) - d$	$- * + a b c d$	$a b + c * d -$

Infix to Postfix Conversion Algorithm

1. Scan the Infix string from left to right.
2. Initialize an empty stack.
3. If the scanned character is an operand, add it to the Postfix string.
4. If the scanned character is an operator and if the stack is empty push the character to stack.
5. If the scanned character is an Operator and the stack is not empty, compare the precedence of the character with the element on top of the stack.
6. If top Stack has higher precedence over the scanned character pop the stack else push the scanned character to stack. Repeat this step until the stack is not empty and top Stack has precedence over the character.
7. Repeat 4 and 5 steps till all the characters are scanned.
8. After all characters are scanned, we have to add any character that the stack may have to the Postfix string.

9. If stack is not empty add top Stack to Postfix string and Pop the stack.
10. Repeat this step as long as stack is not empty.

Conversion to Postfix EXAMPLE:

$$A + (B * C - (D / E - F) * G) * H$$

Stack	Infix	Postfix
Empty	$A + (B * C - (D / E - F) * G) * H$	-
Empty	$+(B * C - (D / E - F) * G) * H$	A
+	$(B * C - (D / E - F) * G) * H$	A
+(	$B * C - (D / E - F) * G) * H$	A
+(	$* C - (D / E - F) * G) * H$	AB
+(*	$C - (D / E - F) * G) * H$	AB
+(*	$- (D / E - F) * G) * H$	ABC
+(-	$(D / E - F) * G) * H$	ABC*
+(- (	$D / E - F) * G) * H$	ABC*
+(- (	$/ E - F) * G) * H$	ABC*D
+(- (/	$E - F) * G) * H$	ABC*D
+(- (/	$- F) * G) * H$	ABC*DE
+(- (-	$F) * G) * H$	ABC*DE/
+(- (-	$F) * G) * H$	ABC*DE/
+(- (-	$) * G) * H$	ABC*DE/F
+(-	$* G) * H$	ABC*DE/F-
+(-*	$G) * H$	ABC*DE/F-
+(-*	$) * H$	ABC*DE/F-G
+	$* H$	ABC*DE/F-G*-

+	H	ABC*DE/F-G*-
*	End	ABC*DE/F-G*-H
Empty	End	ABC*DE/F-G*-H*+

Infix to Prefix Conversion Algorithm

To convert an infix to prefix expression, we use the same algo for infix to postfix to convert Infix to Prefix.

Step 1: Reverse the infix expression i.e $A+B*C$ will become $C*B+A$. Note while reversing each '(' will become ')' and each ')' becomes '('.

Step 2: Obtain the postfix expression of the modified expression i.e $CB*A+$.

Step 3: Reverse the postfix expression. Hence in our example prefix is $+A*BC$.

Infix to prefix conversion

Expression = $(A + B \wedge C) * D + E \wedge 5$

Step 1. Reverse the infix expression.

$5 \wedge E + D * (C \wedge B + A)$

Step 2. Make Every '(' as ')' and every ')' as '('

$5 \wedge E + D * (C \wedge B + A)$

Step 3. Convert expression to postfix form.

Step 2 Expression	Stack	Postfix	Comment
$5 \wedge E + D * (C \wedge B + A)$	Empty	-	Initial
$\wedge E + D * (C \wedge B + A)$	Empty	5	Print
$E + D * (C \wedge B + A)$	^	5	Push
$+ D * (C \wedge B + A)$	^	5E	Push
$D * (C \wedge B + A)$	+	5E^	Pop And Push
$*(C \wedge B + A)$	+	5E^D	Print
$(C \wedge B + A)$	+	5E^D	Push

C^B+A)	+*(	5E^D	Push
^B+A)	+*(	5E^DC	Print
B+A)	+*(^	5E^DC	Push
+A)	+*(^	5E^DCB	Print
A)	+*(+	5E^DCB^	Pop And Push
)	+*(+	5E^DCB^A	Print
End	+	5E^DCB^A+	Pop Until '('
End	Empty	5E^DCB^A++	Pop Every element

Step 4. Reverse the expression.

+ * + A ^ B C D ^ E 5

Algorithm for Prefix to Postfix:

1. Read the Prefix expression in reverse order (from right to left)
2. If the symbol is an operand, then push it onto the Stack
3. If the symbol is an operator, then pop two operands from the Stack
 - 3.1 Create a string by concatenating the two operands and the operator after them.
 - 3.2 string = operand1 + operand2 + operator
 - 3.3 And push the resultant string back to Stack
4. Repeat the above steps until end of Prefix expression.

Lab Task

1. Write a C++ program to convert an Infix expression into Prefix expression. Also check whether Infix expression is mathematically correct or not.
2. Write a C++ program to convert an Infix expression into Postfix expression.
3. Write a C++ program to convert a Prefix expression into Postfix expression.

Conclusion
